

Informe del proyecto SnailPay

Resumen

La aplicación permite registrar una cuenta local, iniciar y cerrar sesión, consultar saldo y estadísticas simuladas, y realizar recargas ficticias mediante una API de Express. El trabajo conserva una estructura sencilla de rutas, componentes y servicios. La funcionalidad está implementada y cuenta con comprobaciones automatizadas; la revisión completa de interacción y presentación en navegador sigue pendiente.

Proceso y decisiones

Se revisaron los requisitos y el código inicial, se completó la API de pagos y después se conectaron los formularios y el dashboard. Se corrigieron validaciones, respuestas por campo, navegación, estilos y gráficos. La última revisión mantuvo los componentes existentes y añadió pruebas de integridad del saldo.

El frontend usa React y TypeScript sobre una plantilla TanStack Start con Router, Vite y Nitro. El backend usa Express y TypeScript. LocalStorage conserva usuario, sesión, saldo y datos ficticios de la última recarga aprobada. No hay conexión con servicios financieros ni base de datos.

Organización

Las rutas del backend llaman a un controlador que obtiene el resultado del servicio; un validador comprueba entradas y un constructor genera las respuestas. El frontend separa acceso, registro, recarga y estadísticas. InputCustom reúne etiquetas, controles y errores. La aplicación del saldo se concentra en la utilidad de cuenta para poder probarla sin un navegador.

Herramientas y apoyo de IA

Se emplean Tailwind CSS, shadcn con Base UI y Recharts. Las tarjetas, campos, botones y contenedor de gráficos parten de esos componentes; las pantallas y la integración se adaptaron al flujo del proyecto. El donut se representa con CSS y las barras con Recharts.

Codex apoyó el análisis, implementación, refactorización limitada, correcciones, pruebas y documentación. Se validaron tipos, lint, compilación y pruebas automatizadas. El autor debe completar su descripción de las adaptaciones personales y revisar que puede explicar el código antes de presentar la entrega.

Funcionalidades y validación

Funcionalidades implementadas

Registro con nombre, correo, contraseña y confirmación; contraseña mínima de ocho caracteres, hash PBKDF2 con sal y sesión local. El usuario comienza con saldo cero. Se recuperan los datos al recargar y se restringe el contenido del dashboard cuando no existe una sesión activa.

El dashboard muestra nombre, correo, saldo, cierre de sesión y recarga. Incluye cuatro apuestas ganadas y dos perdidas en un donut, y seis caracoles con victorias 2, 1, 0, 1, 2 y 0: seis carreras simuladas. La distribución usa una columna en pantallas pequeñas y dos en pantallas amplias.

Integración de pagos

El formulario envía tarjeta, vencimiento, CVV, titular, monto, identificador y correo del usuario. Una aprobación válida guarda el nuevo saldo y lo muestra inmediatamente. Rechazos y errores no suman saldo. Se bloquean envíos mientras se procesa una solicitud y se cancela la espera después de diez segundos.

Las respuestas incluyen id, status, status_detail, transaction_amount, date_created, authorization_code, reference, payer_id, payer_email y tarjeta y CVV ficticios. Los errores contienen campo, código y mensaje. Se acepta un monto positivo, finito y con hasta dos decimales. La fecha fija 12/26 no depende del calendario real en este mock.

Pruebas y motivo de elección

Se ejecutaron 11 pruebas del backend y 6 del frontend, todas aprobadas. Se comprobaron aprobación, rechazo, caída, validaciones, JSON inválido, precisión del monto y comportamiento normal del servicio. En frontend se probaron almacenamiento, sesión, contraseña, datos corruptos, aplicación de aprobaciones, duplicados y fallos al guardar.

Estas pruebas priorizan que el saldo solo cambie ante una aprobación válida y que las respuestas expliquen los errores. TypeScript y lint del frontend pasan y la compilación de producción finalizó correctamente. No se han completado pruebas automáticas de interacción con la interfaz ni una revisión visual final en navegador.

Ejecución y pendientes de entrega

Cómo ejecutar y reproducir resultados

Con Node.js 22.22 o superior, instalar dependencias e iniciar el modo de desarrollo en las carpetas backend y frontend siguiendo sus README. Abrir localhost:3000; la API usa el puerto 3001. Los README también contienen los comandos de pruebas y compilación.

Para aprobar, usar la tarjeta ficticia 1234123412341234, vencimiento 12/26, CVV 543, titular no vacío y monto positivo de hasta dos decimales. Cambiar el CVV a 000 simula rechazo; usar 12 simula formato inválido. Activar el checkbox de caída simula indisponibilidad. Para comprobar un fallo de red, detener el backend; para timeout, demorar una respuesta más de diez segundos en la prueba.

Limitaciones conocidas

La autenticación es local y no ofrece seguridad de servidor. Solo existe una cuenta por navegador; se conserva la última recarga aprobada y no un historial. No se garantiza coordinación transaccional entre recargas simultáneas en varias pestañas. Los datos de tarjeta deben ser siempre ficticios.

Falta comprobar visualmente móvil y escritorio y recorrer registro, acceso, recarga, rechazo, caída, timeout y persistencia en un navegador. Las pruebas unitarias de saldo no sustituyen esa revisión. No se implementaron despliegue público ni propuesta de base de datos, que son opcionales.

Información que debe completar el autor

Tiempo aproximado invertido: pendiente de completar por el autor. No se ha deducido a partir de la conversación.

Repositorio configurado: <https://github.com/edgarlandal/Edgar-1964-API>. No se verificó su visibilidad pública. Debe revisarse el nombre para ajustarlo al formato nombre y cuatro cifras, publicar los cambios y actualizar el enlace si se renombra.

Antes de entregar, completar la aportación personal, confirmar el enlace público, ejecutar una revisión manual final y ajustar las declaraciones de funcionalidades si se detectan problemas. Este informe usa Arial de 10 puntos, no incorpora código ni capturas y tiene tres páginas.